

Project 2 Report

Adan Ruiz and Shantia Ghaninia

Problems Encountered

The main problem for us honestly was having to do this in C++. Memory management and pointer management were the hardest thing for us because we kept hitting seg faults. Another problem for us was creating the graph without being able to use vector or map. Because of this restriction, we had to use a dynamic 2D array to represent the adjacency matrix, which means careful memory management and indexing. Another hard thing was implementing Dijkstra's algorithm without a queue. We had to manually scan for the minimum unvisited vertex at each step, which made it more complex and careful handling of edge cases like unreachable nodes. Also, making sure the output formatting matched the exact spec was hard, since even small differences in spacing or line breaks could make it look wrong.

Known Bugs / Incomplete Implementation

At this time, we see no bugs or incomplete parts. The program correctly reads the input graph, builds the adjacency matrix, computes vertex degrees, finds all odd degree nodes, and runs Dijkstra's from each of those nodes. The implementation has been tested on multiple inputs and has the correct outputs so far.

References

The lecture slides and Dijkstra's algorithm slides were used as the main reference. Also more studying on Dijkstra's algorithm in general was taken from geeks for geeks.

Design Decisions

The graph is represented using a dynamic 2D adjacency matrix array, which allows constant time edge lookups and fits the spec. A separate array is used to track the degree of each vertex, making it efficient to find odd degree nodes. Dijkstra is implemented using arrays for distances and visited nodes, with a helper function to repeatedly select the unvisited vertex with the minimum distance. This avoids restricted data structures while still computing shortest paths. The program starts indexing at 1 to match the input format and simplify mapping between vertices and array indices.